

Sorting 1

"I'd rather attempt to do something great and fail, than to attempt nothing and succeed."

~ Robert H. Schuller

BRIGHT
DROPS
.com



Good

Evening



Content

→ Basics of sorting → Stable / unstable
↳ Inplace sorting

01. Insertion sort

02. Bubble sort

03. Merge two sorted Arrays

04. Mergesort

Sorting: Arranging data in specific order **based on a Parameter**

1 2 3 5 7 → Asc order (parameter → values)

9 6 3 2 1 → Desc order (parameter → values)

1 7 2 9 4 → Asec order (parameter - factors)

fac 1 2 2 3 3

Why sorting? → easy to access the data
→ Searching of data becomes fast

Stable / Unstable sorting → When 2 data points have the same parameter value & their relative order is preserved after the sorting, then it is called stable sorting

sort data in increasing order based on marks

Name	Marks
Mohit	0
Aman	100
Shreesh	45
Kajal	32
Aayush	45
Aditya	92

→ **Stable sorting Algo**

Sort 1	Sort 2
Mohit	Mohit
Kajal	Kajal
Shreesh	Aayush
Aayush	Shreesh
Aditya	Aditya
Aman	Aman

Eg: 1 5 2 6 2

Sort 1 $\rightarrow$ 1 2 2 5 6 $\longrightarrow$ stable sorting

Sort 2 $\rightarrow$ 1 2 2 5 6 $\longrightarrow$ unstable sorting

Inplace sorting algorithm

$\hookrightarrow$ If sorting algo takes $O(1)$ space, then it is considered as inplace sorting algorithm.

01. Given $ar[N]$, first $n-1$ elements are sorted, sort the entire array.

$ar[6] = \{ 2 \quad 6 \quad 10 \quad 14 \quad 20 \quad 4 \}$
 0 1 2 3 4 5

$Sort[] = \{ 2 \quad 4 \quad 6 \quad 10 \quad 14 \quad 20 \}$
 0 1 2 3 4 5

Idea 1 $\rightarrow$ Sort the entire array.

Arrays.sort(ar) $\longrightarrow$ TC: $O(n \log n)$

SC: $O(1)$

$\hookrightarrow \{ 2 \quad 4 \quad 6 \quad 10 \quad 14 \quad 20 \}$

Idea 2 $\rightarrow$ Insert the last ele at its correct position

$$ar[6] = \{ 2 \quad \cancel{6} \quad \cancel{10} \quad \cancel{14} \quad \cancel{20} \quad \cancel{4} \}$$

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ & 4 & \cancel{4} & \cancel{4} & \cancel{4} & 20 \\ & & 6 & 10 & 14 & \end{array}$$

$$Sort[] = \{ 2 \quad 4 \quad 6 \quad 10 \quad 14 \quad 20 \}$$

$$ar[7] = \{ 3 \quad 4 \quad 10 \quad 14 \quad 20 \quad \boxed{24} \quad 5 \}$$

last sorted ele

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ & & 5 & \cancel{5} & \cancel{5} & \cancel{5} & 24 \\ & & & 10 & 14 & 20 & \end{array}$$

j	$ar[j] \leq ar[j+1]$	
5	$24 \leq 5$	X swap
4	$20 \leq 5$	X swap
3	$14 \leq 5$	X swap
2	$10 \leq 5$	X swap
1	$4 \leq 5$	→ break

Complete idea → Iterate from the back & check the adjacent ele, if they are not in correct order, you swap them [Insertion step]

```
for ( j = n-2 ; j ≥ 0 ; j-- )
```

```
    if ( arr[j] > arr[j+1] )
```

```
        swap ( arr, j, j+1 );
```

```
    else break;
```

TC: $O(n)$

SC: $O(1)$

Q Given an arr[], sort it using **insertion steps**

arr[6] : { 10 3 6 8 2 5 }

0 1 2 3 4 5

Step 1 : { 3 10 6 8 2 5 }

Step 2 : { 3 6 10 8 2 5 }

Step 3 : { 3 6 8 10 2 5 }

Step 4 : { 2 3 6 8 10 5 }

Step 5 : { 2 3 5 6 8 10 }

Insertion
Sorting

```
void insertion sort (int [] ar)
```

```
    for (i=1 ; i < N ; i++)
```

```
        // insert ar[i] in sorted part [0...i-1]
```

```
        for (j=i-1 ; j ≥ 0 ; j--) { ←
```

```
            if (ar[j] > ar[j+1]) ←
```

```
                swap (arr, j, j+1) ←
```

```
            else break ; X
```

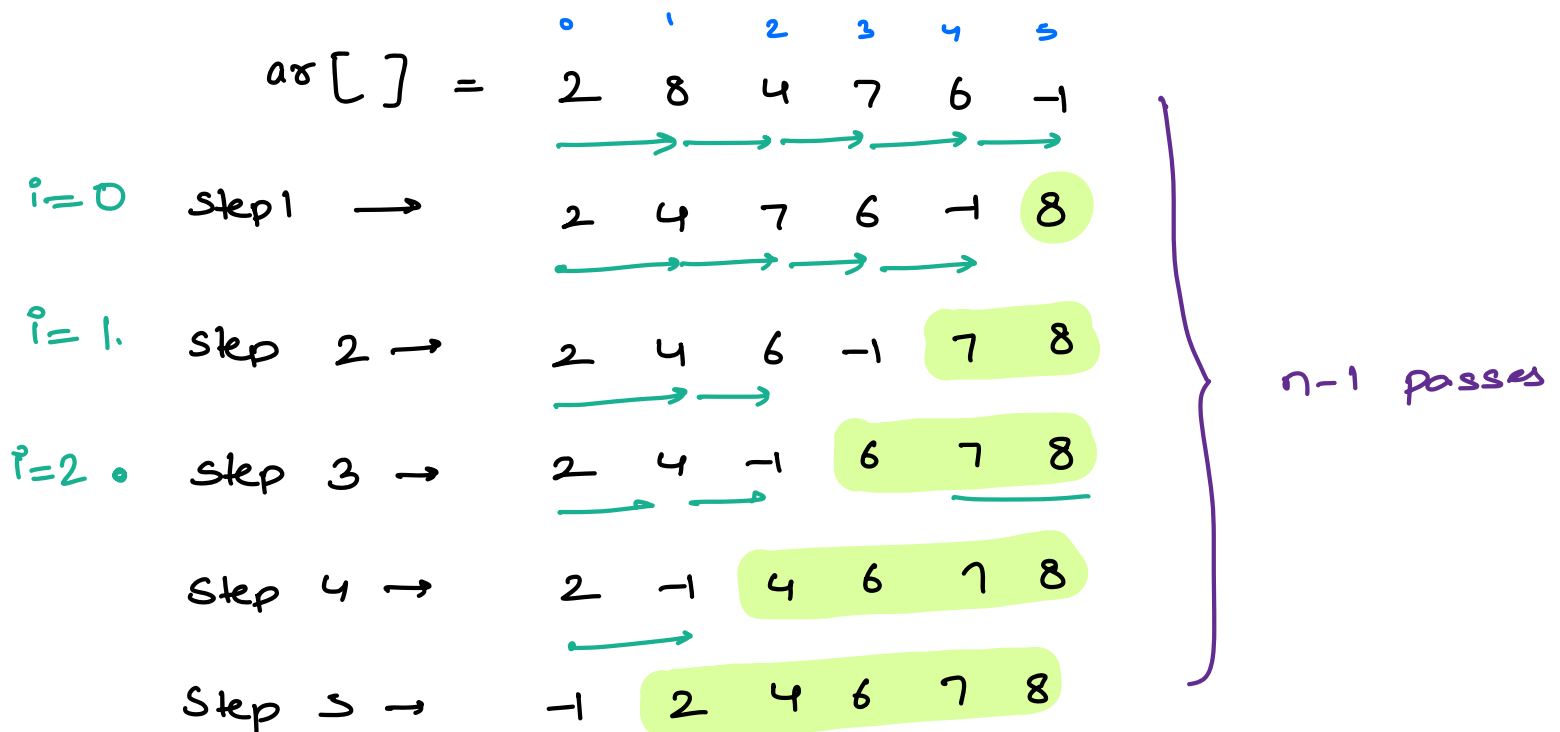
Q = You are going to get a stream of data
& you always have to return a sorted
array

eg: 6 7 3 4 . . .

Sorted = 3 4 6 7

} Stream of
data

02. Given $arr[N]$, we can only swap adjacent elements, sort the array in **increasing order**



```
void bubbleSort (int arr[])
```

```
    for (i=0; i<n-1; i++)
```

```
        for (j=0; j<n-i-1; j++)
```

```
            if (arr[j] > arr[j+1])
```

```
                swap(arr, j, j+1);
```

TC: $O(n^2)$

SC: $O(1)$

$O(1)$
↓
Inplace
sorting
algo

10:19 → 10:28

Ex:- { 4 2 6 7 }

Step 1 → 2 4 6 7

Step 2 = 2 4 6 7 → count the no. of times
we are swapping

↓

○ → sorted

{ 7 6 5 4 2 }

03. Given 2 sorted arrays . $a[N]$, $b[M]$, create $c[]$ which contains overall sorted data

Eg:- $a[4] : \{ 7 \ 10 \ 11 \ 14 \}$

$a[3] : \{ 3 \ 6 \ 10 \}$

$b[3] : \{ 3 \ 8 \ 9 \}$

$b[2] : \{ 5 \ 9 \}$

$c[7] = \{ \underset{0}{3} \ \underset{1}{7} \ \underset{2}{8} \ \underset{3}{9} \ \underset{4}{10} \ \underset{5}{11} \ \underset{6}{14} \}$

$c[5] = \{ 3 \ 5 \ 6 \ 9 \ 10 \}$

$a[3] = \{ \underset{0}{1} \ \underset{1}{2} \ \underset{2}{4} \}$

$b[2] = \{ \underset{0}{1} \ \underset{1}{7} \}$

$c[5] = \{ \underset{0}{1} \ \underset{1}{1} \ \underset{2}{2} \ \underset{3}{4} \ \underset{4}{7} \}$

Idea 1 $\rightarrow$ Add both the arrays in a new array & then sort.

$$\text{Array.sort}(c) = (n+m) \log(n+m)$$

Idea 2 $\rightarrow$ Use two pointer technique

P_1 // copy all remaining elements
↓
 $a[4] : \{ 7 \ 10 \ 11 \ 14 \}$
0 1 2 3

$b[3] : \{ 3 \ 8 \ 9 \}$
0 1 2

↑ // out of array
 P_2

$c[7] = \{ 3 \ 7 \ 8 \ 9 \ 10 \ 11 \ 14 \}$

Idea

{	if ($A[P_1] < B[P_2]$)	→ insert $A[P_1]$ &
		P_1++
{	else	insert $B[P_2]$ & P_2++

int [] merge (int a[n], int b[m])

int [] c = new int [n+m].

P₁ = 0, P₂ = 0, k = 0

while (P₁ < n && P₂ < m)

if (a[P₁] < b[P₂])

c[k] = a[P₁]

P₁ ++;

k ++;

}

else {

c[k] = b[P₂]

P₂ ++;

k ++;

}

}

while (P₁ < n)

c[k] = a[P₁]

P₁ ++;

k ++;

}

TC: O(n+m)

SC: O(1)

// P₂ is out of
the array

while ($P_2 < m$)

$c[k] = b[P_2]$

P_2++ ;

$k++$;

3

P_1 is out of the array

?

04 Given an array of N elements & 3 indices s, m, e & subarray $[s, m]$ is sorted subarray $[m+1, e]$ is sorted

Sort the subarray from $[s, e]$

$\frac{s}{2} \quad \frac{m}{6} \quad \frac{e}{9}$

arr[12] =

0	1	2	3	4	5	6	7	8	9	10	11
4	8	-1	2	6	9	11	3	4	7	13	20

P_1 ↓ P_2 ↓

s m $m+1$ e

$c[e-s+1] =$

-1	2	3	4	6	7	9	11
----	---	---	---	---	---	---	----

arr[12] = { 4 8 -1 2 3 4 6 7 9 11 13 20 }

0 1 2 3 4 5 6 7 8 9 10 11

void merge (int a[n], int s, m, e)

int [] c = new int [e-s+1];

P₁ = s, P₂ = m+1, k = 0

while (P₁ ≤ m && P₂ ≤ e)

if (a[P₁] < a[P₂])

c[k] = a[P₁]

P₁ ++;

k ++;

}

else {

c[k] = a[P₂]

P₂ ++;

k ++;

}

}

while (P₁ ≤ m)

c[k] = a[P₁]

P₁ ++;

k ++;

}

TC: O(n)

SC: O(n)

while ($p_2 \leq e$)

$c[k] = a[p_2]$

p_2++ ;

$k++$;

3

for ($i = 0$; $i < c.length$; $i++$)

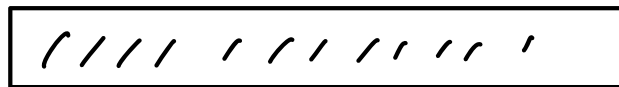
$ar[i+s] = c[i]$

?

?

Q1. Given $ar[N]$, sort it without using inbuilt functions
 $\hookrightarrow N=100$

Case - I

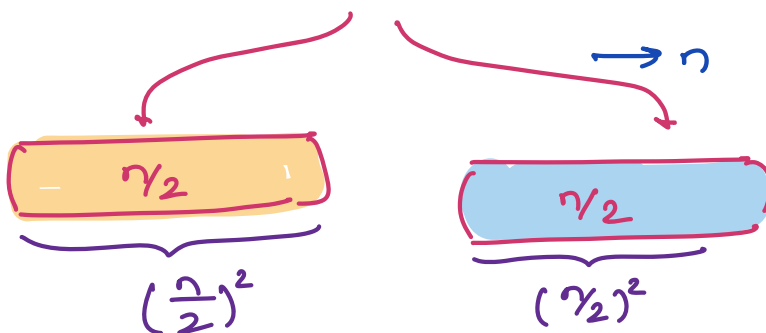


TC
 $O(n^2)$

No. of iterations
 10^7 iterations

Case II

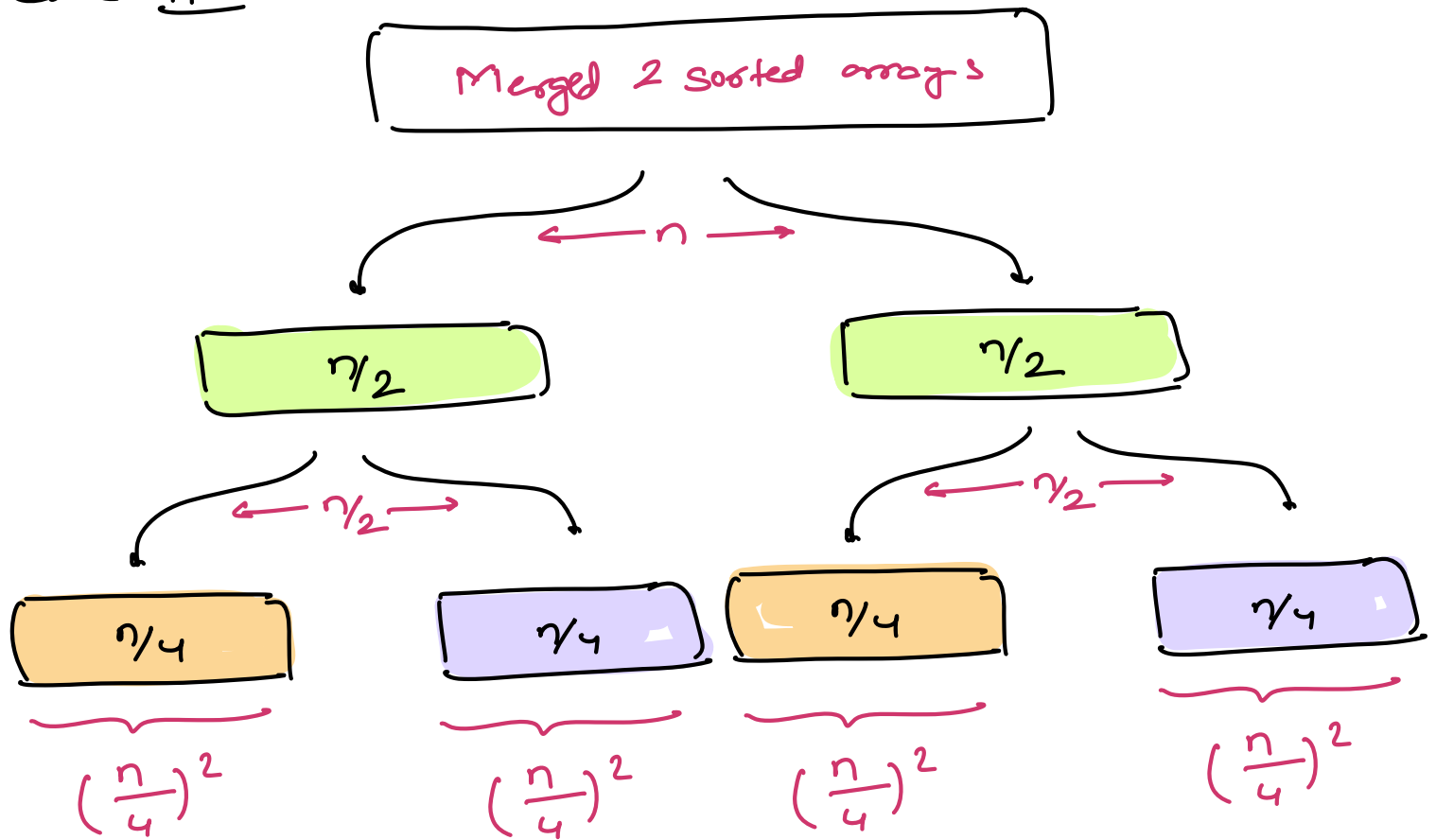
Merge 2 sorted subarr



$$\begin{aligned}
 TC &= 2 * \left(\frac{n}{2}\right)^2 + n \\
 &= \frac{n^2}{2} + n \\
 &= \frac{(100)^2}{2} + 100
 \end{aligned}$$

No. of iterations = 5100

Case - III



$$\text{Total TC} = 4 * \left(\frac{n}{4}\right)^2 + 2n$$

$$= \frac{n^2}{4} + 2n$$

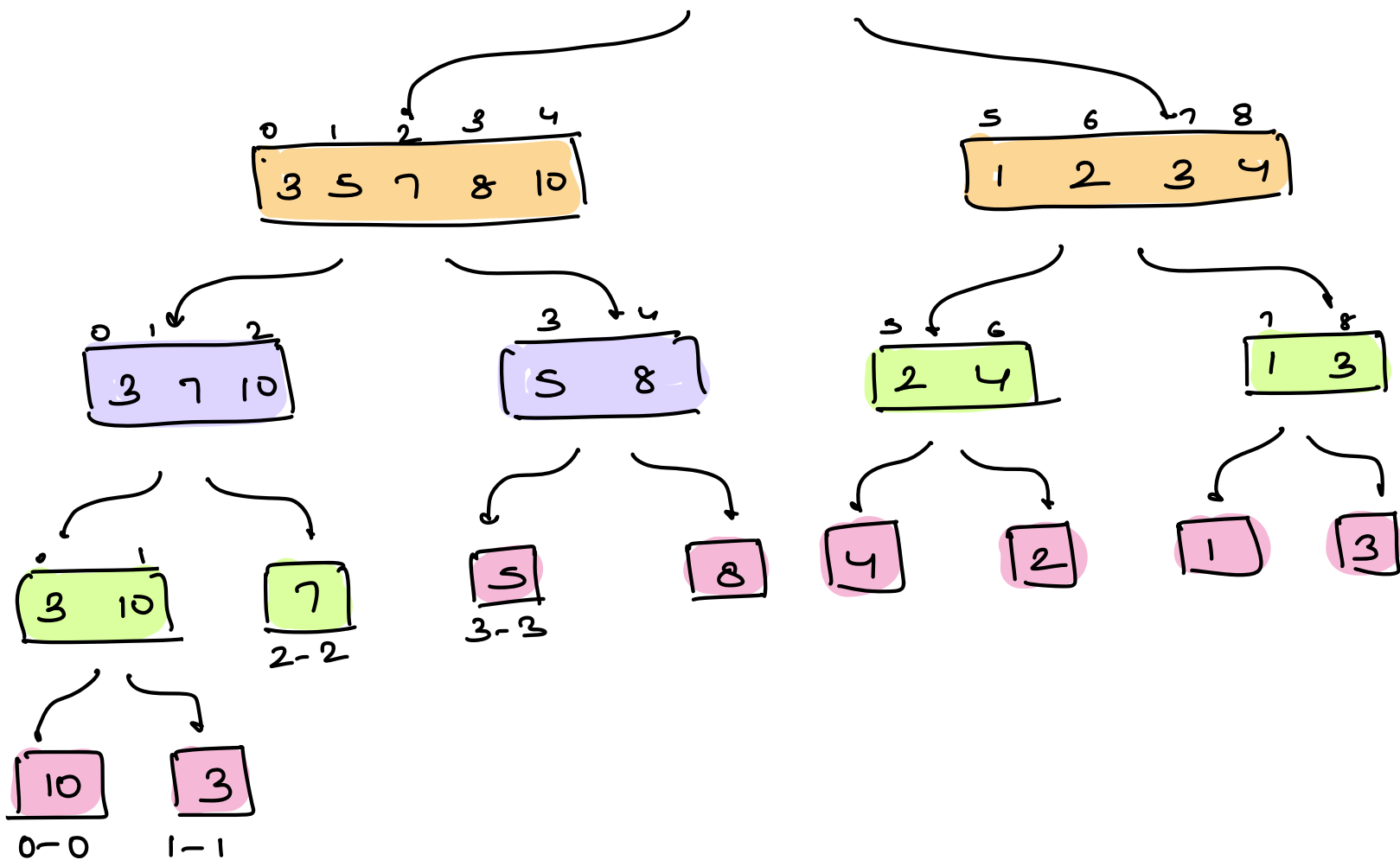
$$= \frac{(100)^2}{4} + 2 * 100 = 2700$$

Idea for Merge Sort

→ Keep on dividing the array until you have single element, start merging the array.

a[9] :

0	1	2	3	4	5	6	7	8
1	2	3	3	4	5	7	8	10



mergesort (int [] ar, int s, int e)

if (s == e) return;

m = (s + e) / 2;

mergesort (ar, s, m)

mergesort (ar, m + 1, e)

merge (ar, s, m, e)

3

TC : → Discuss it in next class

$$\frac{(s+e)}{2} \quad \frac{(0+7)}{2} \Rightarrow 3$$

3 4
3 3

arr = { ²4, ²7, ⁷2, ⁷3 } $\Rightarrow$ { 2 3 4 7 }

0 1 2 3

void insertion sort (int [] ar)

for (i=1 ; i < N ; i++)

// insert ar[i] in sorted part [0...i-1]

for (j=i-1 ; j >= 0 ; j--) { ←

if (ar[j] > ar[j+1]) ←

swap (ar, j, j+1) ←

else break; X

3

2